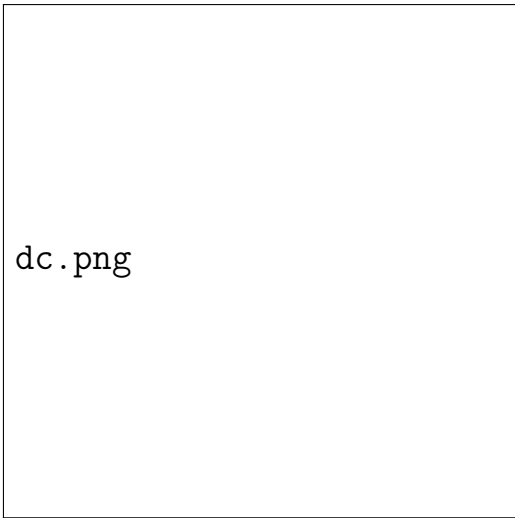




dc.png

Введение

Высшая Школа Цифровой Культуры
Университет ИТМО
dc@itmo.ru

Содержание

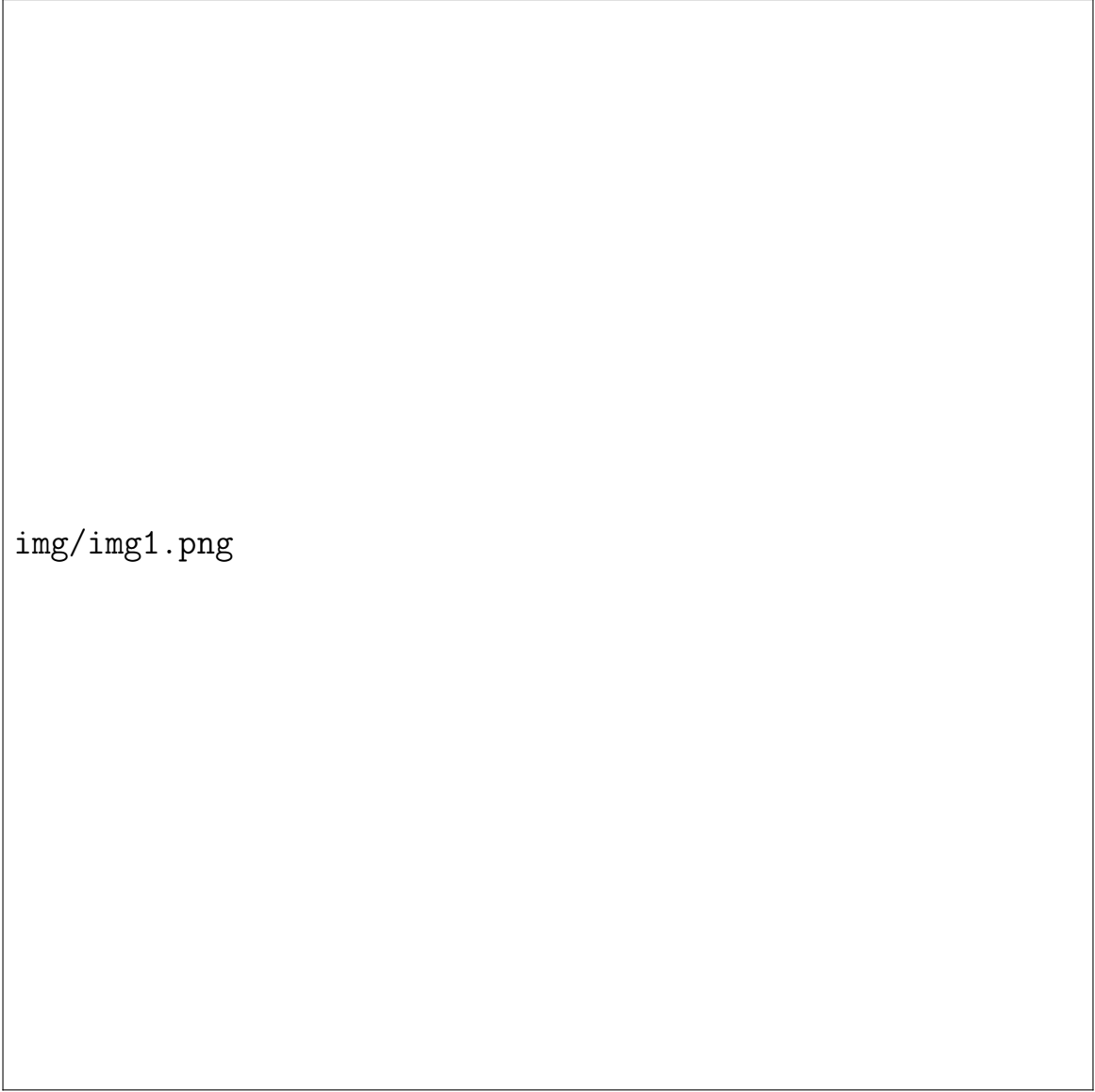
1	Чем занимается классический ML	2
2	Основные разделы ML	4
3	А что же такое DL	6
3.1	Небольшой исторический экскурс	6
3.2	Немного про термин «обучение»	7
3.3	AI effect и основные решаемые задачи	7
3.4	Ключевое отличие DL	10
4	Обучение и переобучение в ML	12
5	Заключение	14

1 Чем занимается классический ML

Здравствуйте, уважаемые слушатели. Приветствуем вас на курсе глубокого обучения. Данная лекция является вводной, в ней мы вспомним о том, что такое машинное обучение, какие задачи оно решает, поймем, какое место в машинном обучении занимают нейронные сети, а также чем подход нейросетей отличается от подходов классического ML. Кроме того, мы напомним базовые принципы обучения и проблемы, которые возникают на практике. Итак, давайте начнем.

В первую очередь начнем с того, что такое машинное обучение и чем оно характеризуется. Машинное обучение – это одно из направлений искусственного интеллекта, призванное, по специально подготовленному набору данных, автоматически выявить в нем зависимости и закономерности, а также построить модель для решения некоей бизнес-задачи. До появления машинного обучения такие закономерности традиционно выявляли с помощью (или посредством) специальных людей – аналитиков, задача которых заключалась в использовании их опыта в заданной области знаний для формализации неких правил или законов, которые в дальнейшем можно было бы использовать. Опыт людей – это хорошо, однако в современных реалиях количество данных стремительно растет, их структура становится все более сложной, и теперь даже супер-профессионалу своей области становится очень сложно охватить весь этот объем информации не то что глазами, но и опытным, натренированным мозгом. Выход из описанной ситуации напрашивается и звучит достаточно просто – передать управление компьютеру, поручив ему анализировать все эти тонны информации. Однако, на практике все оказывается не всегда так радужно.

Итак, резюмируя, вернемся к вопросу того, какую нишу занимает машинное обучение в современном цифровом мире. На сегодняшний момент машинное обучение является средством выявления различных закономерностей в данных. Оно позволяет находить более сложные зависимости, а также определять их границы и накладываемые на них условия более точно. Кроме того, очень часто оказывается, что найденные закономерности контринтуитивны, но правильны, и такие закономерности человеку было бы обнаружить достаточно сложно. Правильно примененные методы машинного обучения очень часто дают более качественный результат, нежели результат, предсказанный профессионалом, так как они оперируют и, что главное, могут одновременно объять большое количество данных, актуальных для конкретной задачи в области. Например, на приведенной иллюстрации алгоритм корректно распознает наложенные друг на друга цифры, которые сложно распознать даже человеку, рисунок 1. В приводимом примере на вход алгоритму подавались черно-белые изображения – верхние в каждом из прямоугольников. Истинные



img/img1.png

Рис. 1: Детектирование цифр

значения написанных цифр (их по две на каждой картинке) указываются в $L : (\cdot, \cdot)$. Те цифры, которые алгоритм должен был выделить, то есть те цифры, которые заставил выделять на картинке человек, указываются в $R : (\cdot, \cdot)$. Интересно посмотреть на результаты в 5 и 6 столбцах (они отмечены звездочками) – там просили найти несуществующие цифры, и алгоритм справился блестяще – ничего не нашел и не обвел, не стал «подгонять» под ответ. В последнем столбце картинок алгоритм просили распознать выделенные (им же, в предыдущем столбце) цифры (P – predict): видно, что цифра 8 была спутана с цифрой 7, а цифра 9 с цифрой 0.

Итак, мы видим, что машины могут научиться решать достаточно сложные задачи. Но не надо считать, что человек при этом ничего не делает: из того, что алгоритм обучается на предоставленных ему данных, очевидным

образом следует то, что от качества этих данных зависит конечный результат, и при неправильно составленном наборе данных, некорректной обработке или неверно выбранном подходе, результаты могут быть абсолютно неудовлетворительными. Так что работа человека-аналитика в современном мире несколько сместилась, но вовсе не упростилась. Именно поэтому аналитик данных, дата сайнтист – одни из самых востребованных на данный момент профессий.

2 Основные разделы ML

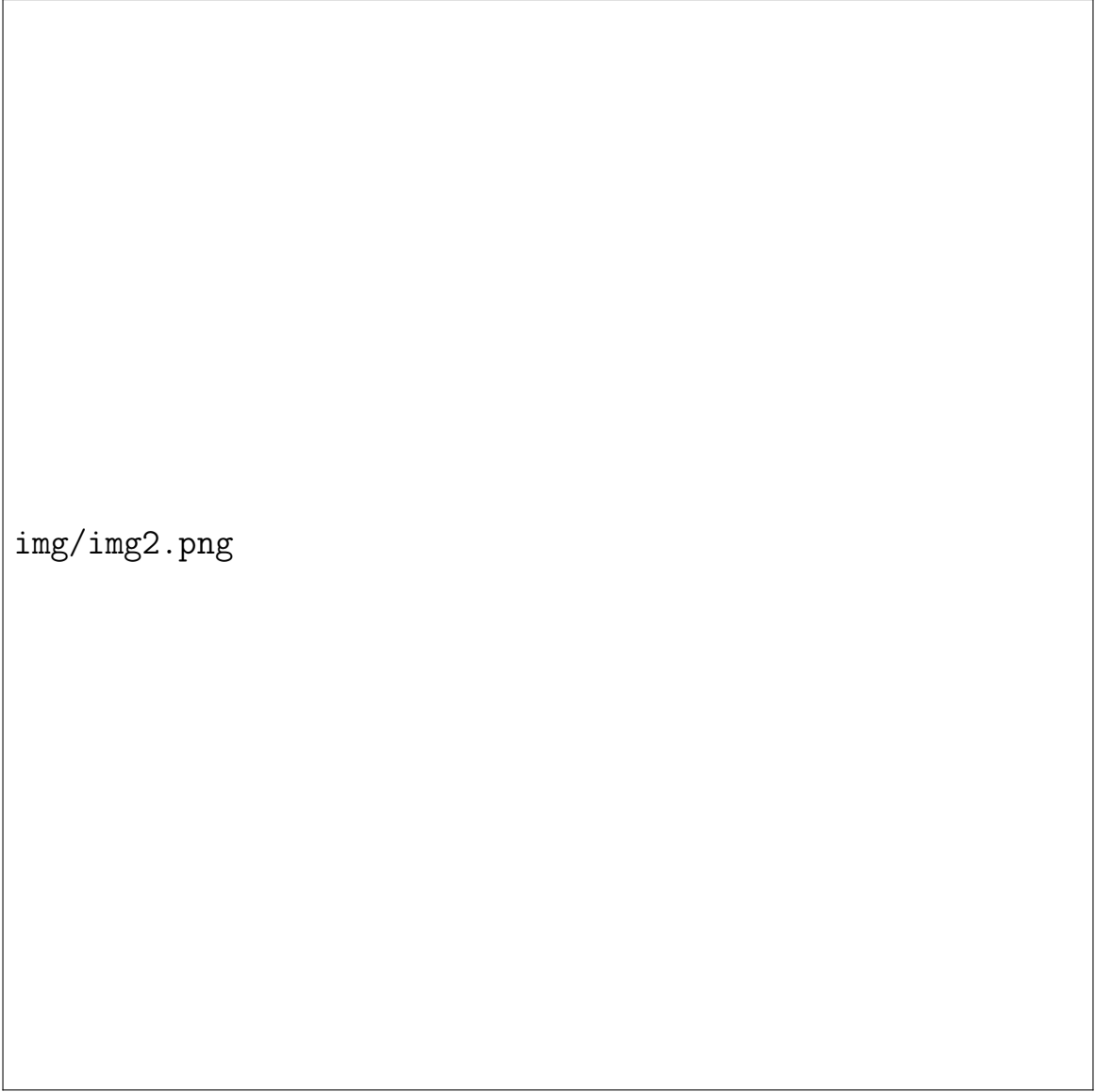
Давайте теперь вспомним, на какие основные ветки делится современное машинное обучение. В первую очередь, это обучение без учителя. Данная ветка занимается построением моделей на основе данных, для которых не предоставлено правильных ответов, или же понятие «правильный ответ» вообще отсутствует. Например, алгоритмы поиска (ассоциативных) правил ищут зависимости в представленных данных и представляют их в формализованном виде, помогая вам, например, найти наиболее часто покупаемые вместе продукты.

Алгоритмы кластеризации используются для объединения данных в схожие группы (кластеры) по совокупности признаков. Этот подход помогает упростить дальнейший анализ данных: можно группировать объекты по различным категориям для рекомендации похожих товаров, можно находить «выбросы» – объекты, не похожие ни на один другой и, возможно, являющиеся некорректными в рассматриваемом наборе данных, и много-много чего еще.

Еще один знаковый представитель ветки обучения без учителя – алгоритмы снижения размерности. Данные алгоритмы позволяют снизить необходимые вычислительные мощности для обучения на представленных данных, помогают визуализировать высокоразмерные наборы данных и даже вычленять их наиболее важные особенности для дальнейшей интеллектуальной обработки.

Еще одной популярной веткой машинного обучения является обучение с учителем. Чаще всего в этой области рассматривают алгоритмы классификации, позволяющие разметить данные по принципу принадлежности к тому или иному заранее заданному классу, и алгоритмы регрессии, позволяющие предсказать значение некоторых (непрерывных) переменных на основе признаков рассматриваемого объекта.

Обучение с частичным привлечением учителя – это ветка машинного обучения, в которой в качестве входных данных используется небольшое количество размеченных данных вместе с большим набором неразмеченных



img/img2.png

Рис. 2: Данные при обучении с частичным привлечением учителя

данных, рисунок 2. Совмещая информацию из обоих наборов, алгоритмы способны более точно разделить классы или предсказать наличие выбросов в данных. Типичным представителем этой области является алгоритм PU-learning, позволяющий построить классификатор на основе небольшого набора (положительно) размеченных элементов и неразмеченного набора.

Еще одной часто упоминаемой веткой является обучение с подкреплением. Алгоритмы обучения с подкреплением призваны принимать решения в ситуациях, когда истинно правильный ответ недоступен. Принцип обучения основан на получении наград за действия, совершенные агентом. Награды могут быть положительными, отрицательными или нейтральными. Задачи в рамках обучения с подкреплением могут ставить весьма по-разному, в зависимости от того, нужно ли анализировать состояние перед принятием реше-

ния, и влияет ли действие агента на состояние среды, рисунок 3. Алгоритмы обучения с подкреплением применяются, в том числе, в беспилотных автомобилях, роботах, рекламе, и много где еще.

Что же, вспомнив основные моменты, связанные с машинным обучением, перейдем к формированию представления о том, что такое глубокое обучение (Deep Learning).

3 А что же такое DL

3.1 Небольшой исторический экскурс

Давайте теперь разберемся с тем, что такое глубокое обучение, и чем оно отличается от классического машинного обучения, задачи которого мы осветили ранее. Начнем же мы с небольшого исторического экскурса.

Сами идеи глубокого обучения витали в воздухе достаточно давно. Первые упоминания возникли еще до двухтысячных годов, но, по причинам малых вычислительных мощностей были «отложены» в долгий ящик. Впрочем, дело не только в вычислительных мощностях. Как мы увидим, нейронные сети – это сложный многопараметрический объект, и зачастую для его корректного обучения нужны большие объемы данных, часто размеченных. А откуда их взять? На тот момент с этим тоже были проблемы.

Революция в мире нейронных сетей произошла примерно в 2005-2006 годах: две отдельно работающие группы под руководством Джеффри Хинтона (университет Торонто) и Йшуа Бенджи (университет Монреаля) научились обучать глубокие нейронные сети. Но прелесть даже не в этом. Хинтон своим подходом улучшил все имевшиеся на тот момент результаты, опирающиеся на стандартные методы машинного обучения, касающиеся распознавания речи.

Впрочем, еще до описанного прорыва исследователи верили в то, что нейронные сети какое-то время будут править миром. Джон Денкер в 1994 году сказал следующую фразу: «Нейронные сети — это второй лучший способ сделать все, что угодно». Первым лучшим способом он считал построение математической модели, заточенной под непосредственно решаемую задачу. Конечно, первый способ кажется очень даже неплохим, но, очевидно немасштабируемым, особенно на то гигантское количество связанных с данными задач, возникающее сейчас.

Итак, хотелось и хочется, конечно, получить универсальный «ключик». И нейронные сети, забегая вперед, роль такого «ключика» неплохо выполняют: одна и та же архитектура с небольшими доработками может использоваться для решения, казалось бы, совершенно разных задач. Частично это и было реализовано Хинтоном — идея предобучения без учителя:

- сначала сеть обучается на большом объеме неразмеченных данных;
- затем сеть дообучается на небольшом объеме размеченных данных.

Идея предобучения используется и по сей день. Например, если мы хотим научить сеть распознавать лица членов семьи, мы можем на первом этапе просто подать ей множество фотографий с лицами. Такой набор данных собрать несложно – фотографий огромное множество в сети Интернет. Затем же, для заточивания сети на решение нашей узкоспециализированной задачи, мы можем дать ей на вход намного менее богатый набор хорошо размеченных данных (например, фотографий членов семьи с метками-именами) и дообучить ее.

3.2 Немного про термин «обучение»

Наверное вы уже заметили, что в разрезе машинного обучения мы постоянно используем слова «обучить», «дообучить», «переобучить». А что вообще означают эти термины? Том Митчелл, основатель первой кафедры по машинному обучению и автор первого учебника по этому предмету, в своей книге пишет: «Компьютерная программа обучается по мере накопления опыта относительно некоторого класса задач T и целевой функции P , если качество решения этих задач (относительно целевой функции P) улучшается с получением нового опыта». Что можно понять из этой фразы? На наш взгляд, ключевое внимание в этой цитате уделяется вовсе не данным (хотя про них тоже есть слово – «опыт»), а целевой функции. И это не случайно: перед решением очередной задачи мы должны в самом начале строго-настрого определить целевую функцию, ведь даже при решении одной и той же задачи смена целевой функции может качественно поменять весь результат.

Скажем, пусть мы хотим научить компьютер читать. Что нам понятно из этой постановки задачи? Компьютер должен просто озвучивать написанное? Или, например, он должен ответить на вопросы по прочитанному? Или показать похожие материалы в Википедии? Или определить авторство текста? Все эти задачи, как вы понимаете, хоть и актуальные, но совершенно разные. И обучать их решению нужно по-разному — отлавливать разные ошибки при обучении — за это и отвечает целевая функция. О выборе тех или иных целевых функций мы будем говорить в процессе развития данного курса.

3.3 AI effect и основные решаемые задачи

Перед тем как перейти к основным задачам, решаемым методами глубокого обучения, хочется озвучить «проблему» этих задач – так называемый AI effect. Оказывается, в мире искусственного интеллекта задача перестает быть «интеллектуальной» ровно в тот момент, когда ее решили. Нельзя не

привести несколько, на наш взгляд, ключевых и отражающих идеологию современных передовых ученых (и философов) в области AI, цитат:

- Pamela Ann McCorduck: It's part of the history of the field of artificial intelligence that every time somebody figured out how to make a computer do something – play good checkers, solve simple but relatively informal problems – there was a chorus of critics to say, 'that's not thinking
- Rodney Brooks: Every time we figure out a piece of it, it stops being magical; we say, 'Oh, that's just a computation
- Fred Reed: A problem that proponents of AI regularly face is this: When we know how a machine does something 'intelligent,' it ceases to be regarded as intelligent. If I beat the world's chess champion, I'd be regarded as highly bright
- Douglas Hofstadter: AI is whatever hasn't been done yet

Как видно, вторая цитата говорит о следующем — как только задача глобально решена, она перестает быть, так скажем «загадочной», и переводится в разряд «счетных». Примерно о том же говорит и третья цитата, возникшая после того, как в 1997 году компьютер IBM обыграл в шахматы чемпиона мира по шахматам Гарри Каспарова: после этого события многие люди стали говорить что компьютер в этой игре не применил никакого интеллекта, лишь метод грубой силы (bruteforce). Поэтому особенно интересно идею AI effect'a, на наш взгляд, описывает последняя цитата: искусственный интеллект — это то, что еще не сделано.

Впрочем, многие задачи получают «вторую жизнь», которая в основном связана с технологическим прогрессом и новыми, более требовательными запросами современности. Например, задача OCR (optical character recognition) — задача распознавания печатного слога, долгое время считавшаяся задачей искусственного интеллекта, достаточно давно решена, рисунок 4 + OCR2.gif. Однако, с появлением портативных мобильных устройств, она снова получила невероятную актуальность. Конечно, это связано с вычислительными мощностями: мобильное устройство не обладает (или не обладало) такими возможностями как, например, мощные стационарные персональные компьютеры, и чтобы решать ту же самую задачу, но с меньшими требованиями к производительности, пришлось серьезно переписать саму архитектуру «решателя», да и, честно говоря, прост-напросто изменить подход.

Есть и совершенно «модельные» задачи. Например, задача распознавания кошек и собак. Задача заключается в следующем: по данной картинке определить, кто на ней присутствует — кошка или собака. Оказывается, это очень сложная задача, рисунок 5. Например, кошек и собак бывает сложно

различить (первые две картинки), скажем, потому что их цвет не контрастирует с цветом фона. Бывает, что сам человек классифицирует то, что перед ним как кошку или собаку только по личному опыту житья с домашними животными (третья и последняя картинки). Иногда, как на четвертой картинке, кошек и собак очень много, и задача становится еще сложнее – определить где-кто (кстати, на картинке, на наш взгляд, очень многие животные похожи). В общем и целом, прямо-таки хорошо решать данную задачу весьма сложно, да и пока что ее хорошо решают немногие, да и это неудивительно. Посмотрите, например, на рисунок 6 – везде ли вы отличите маффин от чихуахуа?

Шутки шутками, но надо понимать, что озвученная задача – это не задача ради задачи. Представьте себе, например, какое-то крупное производство с большим оборотом производимых деталей, каждую из которых нужно проверять на соответствие качеству. Обратите внимание, что эта задача сильно проще: детали обычно весьма однотипны, освещаются они одинаково, фотофиксатор стоит на одном и том же месте, положение деталей на конвейере тоже меняется мало, да и деталь обычно никто не стремится укрыть пледом. Или задача обнаружения какого-то конкретного человека среди всех других – тоже задача, которая сильно проще задачи cat-dog. Так что, если научиться правильно классифицировать кошек и собак, то можно научиться очень хорошо решать и множества похожих, но более простых задач. Конечно, эти задачи решаются при помощи нейронных сетей.

Задачи распознавания голоса – еще одни чисто нейросетевые задачи. Многие ныне существующие помощники как Alice, Siri, Cortana, Alexa, Google Assistant – это яркие представители достижений современных архитектур нейронных сетей. Они не только распознают звук, но и пытаются дать разумный ответ на то, что услышали. Впрочем, наверняка многие из вас прекрасно знакомы с этими инструментами, и их возможности не нуждаются в отдельных комментариях.

Итак, достаточно конкретных примеров. Давайте обобщим. Для начала отметим, что машинное обучение является подобластью наук об искусственном интеллекте, а глубокое обучение, в свою очередь, является подобластью машинного обучения, рисунок 7. В основе глубокого обучения лежит тот же принцип, что и в основе машинного обучения – на основе данных вывести зависимости и генерализовать их, для возможности применения в дальнейшем.

В настоящее время глубокое обучение применяется во многих задачах, заменяя собой классические алгоритмы машинного обучения, по причине более высокой эффективности, а также упрощения построения моделей. Основные области применения таковы:

- Table and recurrent data

- Computer Vision
- NLP
- RL

Давайте скажем несколько слов о каждой области отдельно. Например, с помощью рекуррентных нейронных сетей стало значительно проще анализировать временные зависимости в данных, позволив, скажем, создавать музыку или предсказывать продажи товаров. В области компьютерного зрения глубокое обучение совершило настоящий переворот, позволив работать с картинками напрямую без предварительного выделения признаков. В области анализа текста последние несколько лет ознаменовались крупными успехами в связи с появлением таких алгоритмов глубокого обучения, как attention, и сетей BERT. Также, в области обучения с подкреплением, глубокое обучение достаточно сильно упростило анализ состояний среды и позволило в некоторых областях, таких как игры Atari или Doom, достичь результатов, которые не может достичь человек. В рамках данного курса мы последовательно изучим и коснемся всех этих областей для того, чтобы разобраться и научиться строить решения на основе алгоритмов глубокого обучения. Более того, мы увидим, что часто удобная архитектура нейронной сети оказывается «кросс-платформенной» – небольшая модификация сети позволяет переключить ее с задач CV на, скажем, задачи NLP.

3.4 Ключевое отличие DL

Поговорим, наконец, о ключевом отличии классического машинного обучения от глубокого обучения.

В классическом машинном обучении достаточно важную часть работы с данными занимает так называемый feature engineering (feature extraction) – процесс выделения из данных важных признаков, рекомбинация существующих признаков в новые, или обогащение данных новыми признаками из других источников. Так как классические алгоритмы не знают ничего о природе данных и рассматриваемой области, задача data scientist’a заключается, в частности, в том, чтобы предоставить алгоритму наиболее важные признаки данных, облегчая работу алгоритма, рисунок 8. Инженерия признаков используется почти в каждом случае **настоящего** применения классических алгоритмов машинного обучения и очень сильно влияет на конечный результат работы алгоритма. Именно в этом месте и кроется главное отличие подходов DL.

Итак, глубокое обучение выделяется тем, что предварительная инженерия признаков либо вовсе не производится, либо она сведена к минимуму. В рамках данной области используются такие алгоритмы машинного обучения,

которые способны самостоятельно выделить признаки из подаваемых данных, в частности – нейронные сети. Преимуществ у данного подхода несколько.

Первым, достаточно очевидным преимуществом является то, что инженер или data scientist освобожден от, соответственно, инженерии признаков, что позволяет сократить затрачиваемое на тестирование гипотез или построение модели время.

Вторым преимуществом является то, что признаки, которые модель выделяет самостоятельно, могут быть (и обычно бывают) более сложными, нелинейными, нежели те, что может самостоятельно выделить человек. В рамках обучения алгоритм находит те признаки, которые оказываются наиболее полезными в процессе решения задачи, причем достаточно часто эти признаки бывают как неинтерпретируемы, так и вовсе непонятны людям.

Озвученный подход часто называют feature learning – процесс изучения и самостоятельного нахождения релевантных признаков объекта алгоритмом вместо подготовки их инженером. Кроме экономии времени, данный подход помогает решать более сложные и глобальные задачи без разбиения их на подзадачи, так как, при условии достаточности вычислительных ресурсов и данных, feature learning оказывается более эффективным подходом к созданию признаков.

Основным недостатком данного подхода является то, что сильно возрастает необходимость в вычислительных ресурсах и количестве данных, необходимых для обучения, поэтому для глубокого обучения используются графические ускорители или даже специально разработанные тензорные процессоры.

Продemonстрируем сказанное на примере. Предположим, нам с вами предстоит научиться отличать на картинках жирафов от слонов. Как мы будем это делать? Классический подход предполагает, что мы с вами выделим какие-нибудь признаки, так как алгоритмы, сами по себе, не умеют работать с картинками и пикселями на них. Давайте, например, выделим два признака – высоту и длину животного, а также их соотношение, и все картинки заменим на описанные данные. После это мы с вами сможем успешно построить бинарный классификатор с помощью, например, метода опорных векторов, который сможет как-то разделить эти классы и поможет нам решить задачу.

В отличие от классических алгоритмов, при применении аппарата сверточных нейросетей, на вход достаточно подать картинку с животным и правильный ответ – является ли животное на картинке слоном или жирафом. Получив достаточное количество данных, алгоритм самостоятельно выделит необходимые признаки и, в дальнейшем, сможет определять на картинках как слонов, так и жирафов, не требуя от инженера предварительного обогащения данных информацией о высоте и длине животного. Кроме того, в

случае, если длина и высота животного окажутся плохими признаками для разделения, скажем, при классификации крокодилов и бобров, алгоритм самостоятельно подберет другие признаки, которые будут наиболее полезны конкретно в данной ситуации, например, наличие у животного меха или зубов. Мы еще вернемся к этим примерам позже, при обсуждении конкретных архитектур нейронных сетей.

4 Обучение и переобучение в ML

Теперь абстрактно поговорим о вопросах обучения модели. Для того, чтобы понимать, насколько качественную модель удалось обучить, необходимо правильно ее «измерить». Давайте с вами вспомним, какие два основных подхода используются в машинном обучении для использования данных при обучении и тестировании модели.

Одной из первых идей, которая приходит в голову при обучении модели – это идея проверки ее качества на том же наборе данных, на котором производилось обучение. Однако, этот подход чреват тем, что мы не сможем адекватно настроить обобщающую или генерализующую способность модели, то есть способность правильно выдавать предсказания для новых, ранее невиданных моделью данных. Все потому, что при увеличении количества эпох обучения (грубо говоря, при увеличении количества прогонов тренировочного набора данных через модель для настройки ее параметров), при условии достаточно большой запоминающей способности модели, модель просто-напросто выучит набор данных, который она видит, и подстроится под него, обеспечивая все большую точность на тренировочном наборе данных с каждой новой итерацией, рисунок 10. Вместе с этим, как правило, генерализующая способность модели будет падать, а именно эта способность нам и интересна.

Для того, чтобы адекватно оценивать генерализующую способность модели и ее текущее состояние, можно использовать тестовую часть изначального набора данных.

Что такое тестовая часть набора данных? Помня, что нам необходимо оценить работу модели на неиспользованных ранее данных, часть данных первоначального тренировочного набора выделяется в отдельный, тестовый набор данных, рисунок 11, и не используется в процессе обучения. Так как для корректной оценки модели необходимо, чтобы распределение данных в тренировочной и тестовой выборке совпадало, обычно в тестовую выборку выделяют случайную подвыборку данных из основного набора. После очередной эпохи обучения, на данном тестовом наборе данных проверяется качество модели. Если на какой-то итерации точность на тренировочной выборке продолжает улучшаться, а на тестовой выборке начинает ухудшаться, рисунок

12, это значит, что модель достигла максимально возможной генерализующей способности при текущей архитектуре и гиперпараметрах, и, следовательно, процедуру обучения стоит остановить. Наблюдая за данным графиком, можно оценить также общую степень генерализации модели и скорость ее сходимости.

Благодаря графику метрик на тренировочном и тестовом наборе данных, мы можем не только оценить модель, но и сравнивать разные модели или одну и ту же модель, запущенную с разными гиперпараметрами. Меняя гиперпараметры и наблюдая за графиком и финальным значением метрик, можно подобрать оптимальную модель под решаемую задачу и настроить ее наилучшим образом. Однако, в таком случае наша оценка модели или гиперпараметров оказывается скошенной в сторону более качественной работы на тестовом наборе данных – то есть среди всех моделей и наборов гиперпараметров будут выбраны те, которые наиболее подходят к тестовому набору данных. Получается, что модель дообучается на тестовом наборе данных и не может в дальнейшем гарантировать генерализацию и устойчивую работу на новых данных.

В связи с этим, рекомендуется набор данных делить на три части – тренировочную выборку, валидационную и тестовую, рисунок 11. Тренировочная выборка (обычно около 80% данных) служит для тренировки модели, валидационная (около 10% данных) – для наблюдения за процессом обучения, своевременной остановки и избегания так называемого переобучения, подбором модели и гиперпараметров. После того, как модель и гиперпараметры были подобраны, производится оценка модели на тестовом наборе данных, после чего полученная метрика адекватно отражает качество и генерализующую способность модели. После этого не рекомендуется изменять какие-либо гиперпараметры или менять модель, т.к. иначе снова гиперпараметры модели окажутся подобранными под тестовую часть набора данных.

Еще одним способом оценки качества модели является так называемая кросс-валидация. Данная техника заключается в том, что от набора данных отделяется тестовая часть, а оставшийся набор равномерно (или почти равномерно) случайным образом распределяется по K поднаборам, так называемым фолдам. Далее тренируется K одинаковых моделей, причем для обучения i -той модели используются все фолды, кроме i -го. К примеру, если разделить набор данных на 5 фолдов, как на рисунке 13, то первая модель будет обучаться на фолдах номер 2, 3, 4 и 5, вторая – на фолдах 1, 3, 4, 5, и так далее. Полученные метрики после этого усредняются. Данный подход позволяет выявить и нивелировать неравномерность распределения данных в тренировочном и валидационном наборах данных, так как значения метрик будут кардинально различаться для разных фолдов. Недостатком данного метода является повышенные требования к вычислительной мощности, так

как в данном случае тренируется K моделей вместо одной, что может быть достаточно сложным в случае тяжеловесности модели или недостатка вычислительных ресурсов.

Все описанные действия служат для того, чтобы улучшить генерализующую способность модели и препятствовать так называемому переобучению, рисунок 14. На левой части рисунка видно, что построенная модель недостаточно хорошо описывает даже тренировочный набор данных (на практике – большая ошибка на тренировочных и тестовых данных), на средней части тренировочные данные описываются моделью достаточно хорошо, достигнут оптимум обучения, (на практике – небольшая ошибка на тренировочных, небольшая ошибка на тестовых данных), а на правой – явное переобучение (на практике – мизерная ошибка на тренировочных данных и большая на тестовых). Эти же вопросы можно визуализировать и так, рисунок 15. Практические вопросы обучения моделей мы будем обсуждать в соответствующих лекциях.

5 Заключение

Итак, в этой лекции мы вспомнили, что такое машинное обучение и какие основные задачи оно решает. Кроме того, мы узнали, чем качественно подход DL отличается от подхода классического ML. Теперь пора переходить непосредственно к вопросам того, а что такое нейронная сеть и как она строится. Удачи!

img/img3.png

Рис. 3: Обучение с подкреплением



Рис. 4: OCR



Рис. 5: cat-dog

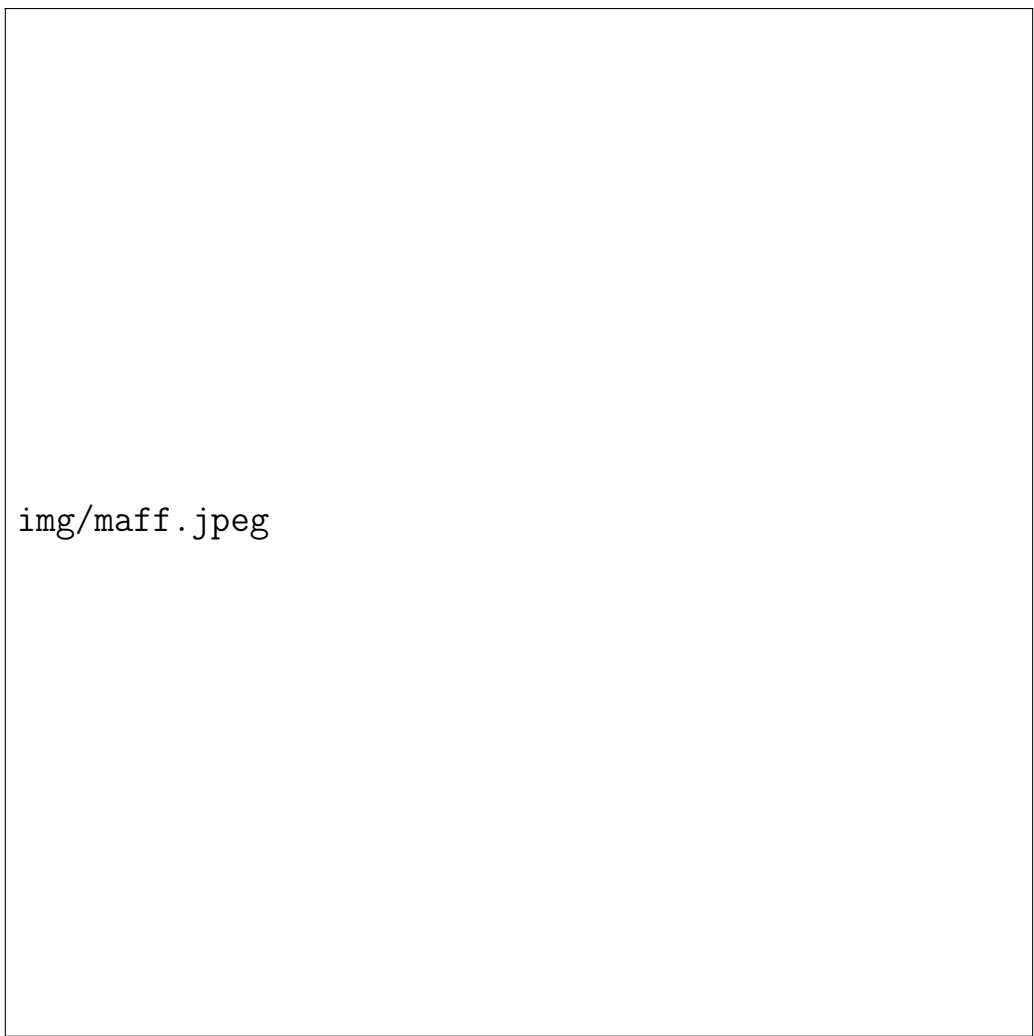


Рис. 6: Чихуахуа или маффин



Рис. 7: AI, ML, DL inference



img/img22.png

Рис. 8: Процессы в классическом МО



Рис. 9: Отличие DL от CML



Рис. 10: Тестирование модели



Рис. 11: Разделение набора данных на части



Рис. 12: Точка окончания обучения



Рис. 13: 5-fold кросс-валидация



Рис. 14: Недо и переобучения



Рис. 15: Недо и переобучения